



Mata Kuliah: **Pengolahan Sinyal Digital (IF3024)**

Tugas: **Tugas Besar**

Nama Anggota:

1. **Fransiskus Xaverius Gunawan (12114010)**
 2. **Arsyadana Estu Aziz (121140068)**
 3. **Tobyanto Putra Mandiri (121140099)**
-

Program Pendeteksi Sinyal Respirasi dan Sinyal rPPG

1 Pendahuluan

1.1 Latar Belakang

Proyek ini merupakan tugas akhir dari mata kuliah "Pengolahan Sinyal Digital IF(3024)" yang dapat digunakan untuk memperoleh sinyal respirasi dan sinyal *remote-photoplethysmography* (rPPG) dari input video web-cam secara *real-time*.

Program ini memperoleh sinyal respirasi dengan menggunakan *pose-landmarker* dari **MediaPipe** untuk menghitung pergerakan bahu saat pengguna melakukan pernapasan. Sedangkan untuk sinyal rPPG, program ini menggunakan *face-detector* dari **MediaPipe** dan algoritma *Plane Orthogonal-to-Skin* (POS) untuk menghitung detak jantung secara non-kontak dengan menganalisis perubahan warna pada wajah pengguna[1].

2 Alat dan Bahan

Untuk menyelesaikan proyek ini, diperlukan beberapa alat dan bahan untuk membuat program yang dapat memperoleh sinyal respirasi dan sinyal rPPG secara non-kontak. Berikut merupakan alat dan bahan yang digunakan:

2.1 Bahasa Pemrograman

Dalam pengembangan program proyek ini, digunakan bahasa pemrograman **Python** karena memiliki dukungan *library* yang umum digunakan untuk proyek komputer visi.

2.2 Library

Berikut merupakan *library* yang digunakan pada pengembangan proyek ini:

1. **OpenCV**: Digunakan untuk memproses gambar dan video.
2. **MediaPipe**: *face-detector* dan *pose-landmarker* dari **MediaPipe** digunakan untuk membuat landmark pada wajah dan bahu pengguna.

3. **PyQt**: Digunakan untuk membuat grafik antarmuka.
4. **NumPy**: Digunakan untuk pengolahan data numerik, seperti penghitungan sinyal dan transformasi data.
5. **os**: Digunakan untuk mengakses fungsi-fungsi sistem operasi
6. **sys**: Digunakan untuk mengakses variabel sistem
7. **requests**: Digunakan untuk mengirimkan permintaan HTTP
8. **tqdm**: Digunakan untuk membuat progress bar
9. **platform**: Digunakan untuk mengakses informasi sistem operasi dari pengguna
10. **scipy**: Membantu dalam analisis sinyal, termasuk ekstraksi fitur dari sinyal respirasi dan rPPG.

2.3 Metode dan Algoritma

Berikut merupakan Metode dan Algoritma yang digunakan pada pengembangan proyek ini:

1. **Ekstraksi Sinyal rPPG**: Menggunakan algoritma *Plane Orthogonal-to-Skin* (POS) berbasis perubahan intensitas warna pada area wajah untuk mendeteksi sinyal detak jantung [1].
2. **Ekstraksi Sinyal Respirasi**: Menggunakan analisis pergerakan bahu dan area wajah untuk mendapatkan pola respirasi pengguna [2].

3 Penjelasan

Program pada proyek ini dibagi menjadi 4 modul, yaitu **main.py**, **check_gpu.py**, **download_model.py**, dan **heart_rate.py**. Berikut adalah penjelasan mengenai alur kerja program yang dikembangkan pada proyek ini:

3.1 Instalasi Library

Agar dapat menjalankan program, pengguna dapat menambahkan beberapa library ke dalam python environment yang akan digunakan. Pengguna dapat mengunduh library yang digunakan menggunakan dua cara, yaitu:

3.1.1 Menggunakan environment.yml

```
1 conda env create -f environment.yml
2
```

Kode 1: Install Library/Pustaka menggunakan environment.yml

3.1.2 Menggunakan requirements.txt

```
1 pip install -r requirements.txt
2
```

Kode 2: Install Library/Pustaka menggunakan requirements.txt

3.2 Modul main.py

3.2.1 Import Library

Library yang digunakan pada program ini adalah sebagai berikut: sys, NumPy, OpenCV, os, subprocess, requests, tqdm, MediaPipe, PyQt, PyQtgraph, platform, SciPy. Selain itu, fungsi `cpu_P0S.py`, `download_model_face_detection`, `download_model_pose_detection`, dan `check_gpu` dipanggil sebagai modul dari folder `utils`.

```
1 import sys
2 import numpy as np
3 import cv2
4 import mediapipe as mp
5 from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QVBoxLayout, QGridLayout
6 from PyQt5.QtCore import QTimer, Qt
7 from PyQt5.QtGui import QImage, QPixmap
8 import pyqtgraph as pg
9 from scipy.signal import butter, filtfilt, find_peaks
10 from mediapipe.tasks import python
11 from mediapipe.tasks.python import vision
12 from utils.heart_rate import cpu_P0S
13 from utils.download_model import download_model_face_detection, download_model_pose_detection
14 from utils.check_gpu import check_gpu
```

Kode 3: Import Library/Pustaka

3.2.2 Kelas HeartRateMonitor

Berikut merupakan penjelasan dari kelas HeartRateMonitor:

```
1 class HeartRateMonitor(QWidget):
2     ...
3
```

Kode 4: Kelas HeartRateMonitor

Kelas HeartRateMonitor berfungsi untuk menampilkan *Graphic User Interface* (GUI) dan menghitung detak jantung dan pernapasan secara real-time. Selain itu, Kelas ini akan menyimpan nilai sinyal rppg dan nilai sinyal pernafasan dari pose detection. Berikut merupakan penjabaran dari kelas HeartRateMonitor:

```
1 def __init__(self):
2     super().__init__()
3     self.initUI()
4
5     ## Menyesuaikan backend dengan sistem operasi pengguna
6     video_backend = cv2.CAP_DSHOW if sys.platform == 'win32' else cv2.CAP_AVFOUNDATION #
7     Menggunakan CAP_DSHOW untuk Windows dan CAP_AVFOUNDATION untuk macOS
8     self.cap = cv2.VideoCapture(0, video_backend)
9     self.fps = self.cap.get(cv2.CAP_PROP_FPS)
10    if self.fps == 0:
11        self.fps = 30
12
13    self.r_signal, self.g_signal, self.b_signal = [], [], []
14    self.resp_signal = []
15    self.face_detector = self.initialize_face_detector()
16    self.pose_landmarker = self.initialize_pose_landmarker()
17    self.features = None
18    self.left_x = None
19    self.top_y = None
20    self.right_x = None
21    self.bottom_y = None
```

```
21 self.timer = QTimer()
22 self.timer.timeout.connect(self.update_frame)
23 self.timer.start(1000 // int(self.fps))
24
```

Kode 5: Metode Konstruktor dari Kelas HeartRateMonitor

Metode ini berguna sebagai konstruktor dari kelas **HeartRateMonitor** untuk menangani, antara lain:

1. Proses inisialisasi antarmuka
2. Pengaturan *backend* kamera (supaya kompatibel dengan sistem operasi pengguna)
3. Inisialisasi variabel penyimpanan
4. Inisialisasi model detector serta landmark dari **MediaPipe**
5. Pengaturan timer

```
1 def initUI(self):
2     self.setWindowTitle('Real-Time Heart Rate and Respiration Monitor')
3     self.setGeometry(100, 100, 1200, 800)
4
5     ## Prepare Plot for Combining with Layout
6     self.video_label = QLabel(self)
7     self.hr_label = QLabel('Heart Rate: -- BPM (Beat Per Minute)', self)
8     self.hr_label.setAlignment(Qt.AlignCenter)
9     self.resp_label = QLabel('Respiration Rate: -- BPM (Breath Per Minute)', self)
10    self.resp_label.setAlignment(Qt.AlignCenter)
11
12    self.plot_widget_hr = pg.PlotWidget()
13    self.plot_widget_hr.setYRange(-3, 3)
14    self.plot_curve_hr = self.plot_widget_hr.plot()
15
16    self.plot_widget_resp = pg.PlotWidget()
17    self.plot_widget_resp.setYRange(-3, 3)
18    self.plot_curve_resp = self.plot_widget_resp.plot()
19
20    ## Making Layout instance and insert the plot into the layout
21    left_layout = QVBoxLayout()
22    left_layout.addWidget(self.hr_label)
23    left_layout.addWidget(self.plot_widget_hr)
24    left_layout.addWidget(self.resp_label)
25    left_layout.addWidget(self.plot_widget_resp)
26
27    right_layout = QVBoxLayout()
28    right_layout.addWidget(self.video_label)
29
30    main_layout = QGridLayout()
31    main_layout.addLayout(left_layout, 0, 0)
32    main_layout.addLayout(right_layout, 0, 1)
33
34    self.setLayout(main_layout)
```

Kode 6: Metode initUI

Metode di atas ini digunakan untuk mempersiapkan kelas PyQt sebagai GUI untuk overlay dari feed live time video dan sinyal (rppg + resp) karena kelas merupakan anak dari QWidget.

```
1 def initialize_face_detector(self):
2     model_path = download_model_face_detection()
3     BaseOptions = mp.tasks.BaseOptions
```

```
4     gpu_checked = check_gpu()
5     if gpu_checked == "NVIDIA":
6         delegate = python.BaseOptions.Delegate.GPU
7     else:
8         delegate = python.BaseOptions.Delegate.CPU
9     options = vision.FaceDetectorOptions(
10         base_options=BaseOptions(
11             model_asset_path=model_path,
12             delegate=delegate
13         )
14     )
15     return vision.FaceDetector.create_from_options(options)
16
```

Kode 7: Metode initialize_face_detector

Metode di atas ini digunakan untuk menginisialisasi fungsi face_detector dari mediapipe untuk proses RPPG.

```
1     def initialize_pose_landmarker(self):
2         model_path = download_model_pose_detection()
3         BaseOptions = mp.tasks.BaseOptions
4         PoseLandmarkerOptions = mp.tasks.vision.PoseLandmarkerOptions
5         VisionRunningMode = mp.tasks.vision.RunningMode
6         gpu_checked = check_gpu()
7
8         if gpu_checked == "NVIDIA":
9             delegate = BaseOptions.Delegate.GPU
10        else:
11            delegate = BaseOptions.Delegate.CPU
12
13        options_image = PoseLandmarkerOptions(
14            base_options=BaseOptions(
15                model_asset_path=model_path,
16                delegate = delegate
17            ),
18            running_mode=VisionRunningMode.IMAGE,
19            num_poses=1,
20            min_pose_detection_confidence=0.5,
21            min_pose_presence_confidence=0.5,
22            min_tracking_confidence=0.5,
23            output_segmentation_masks=False
24        )
25
26        return mp.tasks.vision.PoseLandmarker.create_from_options(options_image)
```

Kode 8: Metode initialize_pose_landmarker

Metode di atas ini digunakan untuk menginisialisasi fungsi pose_detection dari mediapipe untuk proses pendeteksian sinyal respirasi pengguna.

```
1     def update_frame(self):
2         ret, frame = self.cap.read()
3         if not ret:
4             return
5
6         frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
7         mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=frame_rgb)
8         detection_result = self.face_detector.detect(mp_image)
9
10        ## Mediapipe Face Detection
```

```

11     if detection_result.detections:
12         for detection in detection_result.detections:
13             bbox = detection.bounding_box
14             h, w, _ = frame.shape
15             x, y = int(bbox.origin_x), int(bbox.origin_y)
16             width, height = int(bbox.width), int(bbox.height)
17
18             forehead_x = x + width // 4
19             forehead_y = y // 2 + 55
20             forehead_width = width // 2
21             forehead_height = height // 5
22
23             roi_forehead = frame[forehead_y:forehead_y + forehead_height, forehead_x:forehead_x
+ forehead_width]
24             cv2.rectangle(frame, (forehead_x, forehead_y), (forehead_x + forehead_width,
forehead_y + forehead_height), (0, 255, 0), 2)
25
26             self.r_signal.append(np.mean(roi_forehead[:, :, 0]))
27             self.g_signal.append(np.mean(roi_forehead[:, :, 1]))
28             self.b_signal.append(np.mean(roi_forehead[:, :, 2]))
29
30             ## Mediapipe Pose Detection with Optical Flow
31             if self.features is None:
32                 # Initialize ROI and feature detection
33                 self.initialize_features(frame)
34
35             """
36             Proses paling kompleks, dimana akan dilakukan pengecekan terhadap features optical flow (10
buah) dan jika tidak ada features maka akan ditrack ulang oleh mediapipe. Meskipun dengan Gray
Image tetapi model ini sangat berat untuk pose detection sehingga cukup sulit untuk
diptimalisasi pada momen-momen awal ketika belum ditentukan optical flownya. Sehingga
diterapkannya Tracking Failure in case tidak ada features.
37             """
38             frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
39             if len(self.features) > 10:
40                 new_features, status, error = cv2.calcOpticalFlowPyrLK(self.old_gray, frame_gray, self.
features, None, **self.lk_params)
41                 good_old = self.features[status == 1]
42                 good_new = new_features[status == 1]
43                 mask = np.zeros_like(frame)
44                 for i, (new, old) in enumerate(zip(good_new, good_old)):
45                     a, b = new.ravel()
46                     c, d = old.ravel()
47                     mask = cv2.line(mask, (int(a), int(b)), (int(c), int(d)), (0, 255, 0), 2)
48                     frame = cv2.circle(frame, (int(a), int(b)), 3, (0, 255, 0), -1)
49                 frame = cv2.add(frame, mask)
50                 if len(good_new) > 0:
51                     avg_y = np.mean(good_new[:, 1])
52                     self.resp_signal.append(avg_y)
53                     self.features = good_new.reshape(-1, 1, 2)
54                     self.old_gray = frame_gray.copy()
55             else:
56                 # Reinitialize features if tracking fails
57                 self.initialize_features(frame)
58
59             # Draw ROI rectangle for chest area
60             cv2.rectangle(frame, (self.left_x, self.top_y), (self.right_x, self.bottom_y), (0, 0, 255),
2)
61
62             """
63             Proses untuk kalkulasi nilai RPPG dan Resp lalu di set sebagai sinyal oleh PyQt Graph.
Setelah itu, akan ditampilkan video live feed dan ditempel ke PyQt.

```

```

64     """
65     if len(self.r_signal) >= self.fps * 10: # 10 seconds
66         rgb_signals = np.array([self.r_signal, self.g_signal, self.b_signal])
67         rgb_signals = rgb_signals.reshape(1, 3, -1)
68         rppg_signal = cpu_POS(rgb_signals, fps=self.fps)
69         rppg_signal = rppg_signal.reshape(-1)
70
71         filtered_signal = self.bandpass_filter(rppg_signal, 0.75, 3.0, self.fps, order=5)
72         normalized_signal = (filtered_signal - np.mean(filtered_signal)) / np.std(
73         filtered_signal)
74         smoothed_signal = self.moving_average(normalized_signal, window_size=int(self.fps/2))
75
76         peaks, _ = find_peaks(smoothed_signal, distance=self.fps/2)
77         peak_intervals = np.diff(peaks) / self.fps
78         heart_rate = 60.0 / np.mean(peak_intervals) if len(peak_intervals) > 0 else 0
79
80         self.hr_label.setText(f'Heart Rate: {heart_rate:.2f} BPM (Beat Per Minute)')
81         self.plot_curve_hr.setData(smoothed_signal)
82         self.r_signal, self.g_signal, self.b_signal = [], [], []
83
84     if len(self.resp_signal) >= self.fps * 10: # 10 seconds for respiration rate
85         resp_signal = np.array(self.resp_signal)
86         filtered_resp_signal = self.bandpass_filter(resp_signal, 0.1, 0.5, self.fps, order=5)
87         normalized_resp_signal = (filtered_resp_signal - np.mean(filtered_resp_signal)) / np.
88         std(filtered_resp_signal)
89         smoothed_resp_signal = self.moving_average(normalized_resp_signal, window_size=int(self
90         .fps/2))
91
92         resp_peaks, _ = find_peaks(smoothed_resp_signal, distance=self.fps)
93         resp_intervals = np.diff(resp_peaks) / self.fps
94         respiration_rate = 60.0 / np.mean(resp_intervals) if len(resp_intervals) > 0 else 0
95
96         self.resp_label.setText(f'Respiration Rate: {respiration_rate:.2f} BPM (Breath Per
97         Minute)')
98         self.plot_curve_resp.setData(smoothed_resp_signal)
99         self.resp_signal = []
100
101     # Convert frame to RGB for displaying in video_label
102     frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
103     image = QImage(frame_rgb.data, frame_rgb.shape[1], frame_rgb.shape[0], QImage.Format_RGB888
104     )
105     self.video_label.setPixmap(QPixmap.fromImage(image))

```

Kode 9: Metode update_frame

Metode di atas ini digunakan untuk mengambil frame pengguna lalu ditentukan beberapa proses seperti deteksi wajah dengan mediapipe dan pose detection, lalu menentukan bounding box untuk proses selanjutnya. Untuk sinyal RPPG ditetapkan daerah dahi sebagai ROI, dan untuk sinyal respirasi ditetapkan daerah sekitar baru sebagai ROI. Untuk Pose Detection akan diterapkan Optical Flow untuk mengurangi kinerja beban tracking setiap frame.

```

1 def initialize_features(self, frame):
2     roi_coords = get_initial_roi(frame, self.pose_landmarker)
3     self.left_x, self.top_y, self.right_x, self.bottom_y = roi_coords
4     old_frame = frame.copy()
5     old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
6     roi_chest = old_gray[self.top_y:self.bottom_y, self.left_x:self.right_x]
7     self.features = cv2.goodFeaturesToTrack(roi_chest, maxCorners=50, qualityLevel=0.2, minDistance
8     =5, blockSize=3)
9     if self.features is None:

```

```
9         raise ValueError("No features found to track!")
10     self.features = np.float32(self.features)
11     self.features[:, :, 0] += self.left_x
12     self.features[:, :, 1] += self.top_y
13     self.old_gray = old_gray
14     self.lk_params = dict(winSize=(15, 15), maxLevel=2, criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03))
```

Kode 10: Metode initialize_features

Metode di atas ini digunakan untuk mendapatkan nilai *features* untuk keperluan *optical flow* dan membuat object Lucas Kanade sebagai argument dari *optical flow* itu sendiri.

```
1     def bandpass_filter(self, signal, lowcut, highcut, fs, order=5):
2         nyquist = 0.5 * fs
3         low = lowcut / nyquist
4         high = highcut / nyquist
5         b, a = butter(order, [low, high], btype='band')
6         y = filtfilt(b, a, signal)
7         return y
8
```

Kode 11: Metode bandpass_filter

Metode di atas ini digunakan untuk melakukan *bandpass filters* dengan parameter dan tipe datanya sebagai berikut ini:

1. *signal: array* = sinyal target filtering
2. *lowcut: float* = batas bawah frekuensi
3. *highcut : float* = batas atas frekuensi
4. *fs : int* = nilai *sampling rate*
5. *order : int* = tingkat pengurangan amplitudo / frekuensi

```
1     def moving_average(self, signal, window_size):
2         return np.convolve(signal, np.ones(window_size)/window_size, mode='valid')
3
```

Kode 12: Metode moving_average

Metode ini berfungsi untuk melakukan moving average pada sinyal (melihat tren sinyal rppg dan resp). Penjelasan dari nama dan data tipe parameter ada sebagai berikut ini:

1. *signal: array* = sinyal target
2. *window_size: int* = ukuran window untuk moving average

```
1     def closeEvent(self, event):
2         self.cap.release()
3         cv2.destroyAllWindows()
4         event.accept()
5
```

Kode 13: Metode closeEvent

Metode turunan dari kelas QWidget untuk melepaskan resource yang tidak dibutuhkan lagi.

3.2.3 Fungsi get_initial_roi

Berikut merupakan penjelasan dari fungsi get_initial_roi:

```
1 def get_initial_roi(image, landmarker, x_size=100, y_size=30, shift_x=0, shift_y=-30):
2
3     image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Mengubah warna BGR ke RGB
4     height, width = image.shape[:2] # Mengambil dimensi frame webcam
5
6     # Membuat gambar MediaPipe dari frame webcam
7     mp_image = mp.Image(
8         image_format=mp.ImageFormat.SRGB,
9         data=image_rgb
10    )
11
12    # Mendeteksi pose dari frame webcam
13    detection_result = landmarker.detect(mp_image)
14
15    if not detection_result.pose_landmarks:
16        raise ValueError("No pose detected in first frame!")
17
18    # Mendeteksi tubuh pengguna dari landmark pertama
19    landmarks = detection_result.pose_landmarks[0]
20
21    # Mengambil landmark bahu kiri dan kanan
22    left_shoulder = landmarks[11]
23    right_shoulder = landmarks[12]
24
25    # Menghitung posisi tengah dari bahu kiri dan bahu kanan
26    center_x = int((left_shoulder.x + right_shoulder.x) * width / 2)
27    center_y = int((left_shoulder.y + right_shoulder.y) * height / 2)
28
29    # Mengaplikasikan shift terhadap titik tengah
30    center_x += shift_x
31    center_y += shift_y
32
33    # Menghitung batasan ROI berdasarkan posisi tengah dan ukuran ROI
34    left_x = max(0, center_x - x_size)
35    right_x = min(width, center_x + x_size)
36    top_y = max(0, center_y - y_size)
37    bottom_y = min(height, center_y + y_size)
38
39    # Mevalidasi ukuran ROI
40    if (right_x - left_x) <= 0 or (bottom_y - top_y) <= 0:
41        raise ValueError("Invalid ROI dimensions")
42
43    return (left_x, top_y, right_x, bottom_y)
```

Kode 14: Metode get_initial_roi

Fungsi di atas ini berguna untuk Mengambil *Region of Interest* (ROI) dari webcam untuk mendeteksi sinyal respirasi berdasarkan pergerakan posisi bahu pasien. Berikut merupakan perincian dari argumen dan data tipe dari fungsi get_initial_roi:

1. *image*: *np.ndarray* = Frame dari webcam
2. *pose_landmarker*: *task* = MediaPipe pose detector yang sudah didownload
3. *x_size*: *int* = Ukuran ROI pada sumbu x
4. *y_size*: *int* = Ukuran ROI pada sumbu y
5. *shift_x*: *int* = Pergeseran ROI pada sumbu x (dimana nilai positif akan menggeser ROI ke kanan dan sebaliknya)

6. *shift_y: int* = Pergeseran ROI pada sumbu y (dimana nilai positif akan menggeser ROI ke bawah dan sebaliknya)
7. Mengembalikan *tuple* yang berupa koordinat dari ROI (*left_x, top_y, right_x, bottom_y*)

3.2.4 Eksekusi Program dengan Fungsi Main

Modul `main.py` akan dieksekusi sebagai program utama pada proyek tugas besar ini.

```
1 if __name__ == '__main__':
2     app = QApplication(sys.argv)
3     ex = HeartRateMonitor()
4     ex.show()
5     sys.exit(app.exec_())
```

Kode 15: Fungsi Main

Berikut merupakan penjelasan dari fungsi di atas ini:

1. `app = QApplication(sys.argv)` digunakan untuk menginstansiasi `QApplication` yang baru. Pengelolaan aliran kontrol dan pengaturan utama aplikasi GUI juga diatur pada baris kode ini.
2. `ex = HeartRateMonitor()` digunakan untuk menginstansiasi jendela aplikasi utama.
3. `ex.show()` digunakan untuk membuat jendela terlihat pada layar pengguna.
4. `sys.exit(app.exec_())` digunakan untuk memulai perulangan utama dari program dan memastikan bahwa program dapat dihentikan sewaktu ditutup oleh pengguna.

3.3 Modul `check_gpu.py`

Pada Program ini, modul `check_gpu.py` digunakan untuk memeriksa apabila pengguna sedang menggunakan *Graphical Processing Unit* (GPU) atau MLX (Arsitektur Apple Silicon). Jika tersedia, modul akan memerintahkan program untuk menggunakan GPU atau MLX sebagai unit pemrosesan.

3.3.1 Import Library

Berikut merupakan penjelasan dari Import Library pada modul `check_gpu.py`.

```
1 import platform
2 import subprocess
```

Kode 16: Import Library untuk `check_gpu.py`

Modul `check_gpu.py` hanya menggunakan dua pustaka yaitu: `platform` dan `subprocess`. Kedua pustaka ini berguna untuk mengakses informasi terkait sistem operasi yang digunakan pengguna.

3.3.2 Fungsi `check_gpu`

Berikut merupakan penjelasan dari fungsi `check_gpu`.

```
1 def check_gpu():
2     system = platform.system()
3     print(f"System: {system}")
4     # Memeriksa apakah sistem adalah Windows atau Linux
5     if system == "Linux" or system == "Windows":
6         try:
7             nvidia_output = subprocess.check_output(['nvidia-smi']).decode('utf-8')
8             return "NVIDIA"
9         except (subprocess.CalledProcessError, FileNotFoundError):
10            return "CPU"
```

```
11
12 # Memeriksa apakah sistem adalah macOS dengan arsitektur Apple Silicon
13 elif system == "Darwin": # macOS
14     try:
15         cpu_info = subprocess.check_output(['sysctl', '-n', 'machdep.cpu.brand_string']).decode
16         ('utf-8').strip()
17         print(f"CPU: {cpu_info}")
18         if "Apple" in cpu_info: # Ini mencakupi semua chip buatan Apple (M1/M2/M3)
19             return "MLX"
20     except subprocess.CalledProcessError:
21         pass
22     return "CPU" # Menggunakan CPU sebagai default
```

Kode 17: Fungsi check_gpu

Fungsi di atas ini berguna untuk memeriksa ketersediaan GPU pada sistem. Fungsi akan mengembalikan nilai *string*;

1. "NVIDIA" apabila pengguna menggunakan sistem dengan *Graphical Processing Unit* (GPU)
2. "MLX" apabila pengguna menggunakan sistem dengan Apple Silicon
3. "CPU" sebagai nilai *default* dimana pengguna tidak menggunakan baik GPU ataupun Apple Silicon.

3.4 Modul download_model.py

3.4.1 Import Library

Berikut merupakan penjelasan dari pustaka yang digunakan pada modul download_model.py.

```
1 import os
2 import requests
3 from tqdm import tqdm
4
```

Kode 18: Import Library pada modul download_model.py

Modul download_model.py hanya menggunakan 3 pustaka, yaitu: os, requests, dan tqdm.

3.4.2 Fungsi download_model_face_detection

Berikut merupakan penjelasan dari fungsi pada download_model_face_detection:

```
1 def download_model_face_detection():
2     # Membuat direktori model jika belum ada
3     model_dir = "models"
4     os.makedirs(model_dir, exist_ok=True)
5
6     url = "https://storage.googleapis.com/mediapipe-models/face_detector/blaze_face_short_range/
7         float16/latest/blaze_face_short_range.tflite"
8     filename = os.path.join(model_dir, "face_detector.task")
9
10    # Memeriksa apabila file sudah terbuat dan tidak kosong
11    if os.path.exists(filename) and os.path.getsize(filename) > 0:
12        print(f"File model {filename} sudah terunduh. Melewati proses pengunduhan...")
13        return filename
14
15    # Proses pengunduhan dengan progress bar (yang menggunakan library tqdm)
16    try:
17        print(f"Mengunduh model face detector ke direktori: {filename}...")
18        response = requests.get(url, stream=True)
```

```

18     response.raise_for_status()
19     total_size = int(response.headers.get('content-length', 0))
20     block_size = 1024
21
22     with open(filename, 'wb') as f, tqdm(
23         total=total_size,
24         unit='iB',
25         unit_scale=True,
26         unit_divisor=1024,
27     ) as pbar:
28         for data in response.iter_content(block_size):
29             size = f.write(data)
30             pbar.update(size)
31
32     # Verify downloaded file
33     if os.path.getsize(filename) == 0:
34         raise ValueError("File yang terunduh kosong")
35
36     print("Proses pengunduhan model face detector berhasil!")
37     return filename
38
39 except Exception as e:
40     print(f"Terdapat error dalam proses pengunduhan model face detector: {e}")
41     if os.path.exists(filename):
42         os.remove(filename) # Clean up partial download
43     raise

```

Kode 19: fungsi download_model_face_detection

Fungsi di atas ini berguna untuk mengunduh [model face_detector](#) dari MediaPipe. Model ini digunakan untuk mendeteksi muka pengguna dalam video.

3.4.3 Fungsi download_model_pose_detection

Berikut merupakan penjelasan dari fungsi pada download_model_pse_detection:

```

1 def download_model_pose_detection():
2     model_dir = "model"
3     os.makedirs(model_dir, exist_ok=True)
4
5     url = "https://storage.googleapis.com/mediapipe-models/pose_landmarker/pose_landmarker_heavy/
6         float16/latest/pose_landmarker_heavy.task"
7     filename = os.path.join(model_dir, "pose_landmarker.task")
8
9     if os.path.exists(filename) and os.path.getsize(filename) > 0:
10         print(f"File model {filename} sudah terunduh. Melewati proses pengunduhan...")
11         return filename
12
13     try:
14         print(f"Mengunduh model pose landmarker ke direktori: {filename}...")
15         response = requests.get(url, stream=True)
16         response.raise_for_status()
17
18         total_size = int(response.headers.get('content-length', 0))
19         block_size = 1024
20
21         with open(filename, 'wb') as f, tqdm(
22             total=total_size,
23             unit='iB',
24             unit_scale=True,
25             unit_divisor=1024,
26         ) as pbar:
27             for data in response.iter_content(block_size):

```

```

27         size = f.write(data)
28         pbar.update(size)
29
30         if os.path.getsize(filename) == 0:
31             raise ValueError("File yang terunduh kosong")
32
33         print("Proses pengunduhan model pose landmarker berhasil!")
34         return filename
35
36     except Exception as e:
37         print(f"Terdapat error dalam proses pengunduhan model face detector: {e}")
38         if os.path.exists(filename):
39             os.remove(filename)
40         raise

```

Kode 20: Fungsi download_model_pose_detection

Fungsi di atas ini berguna untuk mengunduh [model pose landmarker](#) dari MediaPipe. Model ini digunakan untuk mendeteksi pose tubuh pengguna dalam video (khususnya dalam bagian bahu / torso atas).

3.5 Modul heart_rate.py

3.5.1 Import Library

Berikut merupakan penjelasan dari import library pada modul heart_rate.py

```

1 import numpy as np
2 import cv2
3 import scipy.signal as signal
4

```

Kode 21: Import Library pada modul heart_rate.py

Modul heart_rate.py hanya menggunakan 3 pustaka yaitu: numpy, OpenCV, dan SciPy. 3 pustaka ini berguna dalam perhitungan algoritma *Plane Orthogonal-to-Skin* (POS).

3.5.2 Fungsi cpu_POS

Berikut merupakan penjelasan dari fungsi cpu_POS pada modul heart_rate.py:

```

1 def cpu_POS(signal, fps):
2     eps = 10**-9
3     X = signal
4     e, c, f = X.shape
5     w = int(1.6 * fps)
6
7     P = np.array([[0, 1, -1], [-2, 1, 1]])
8     Q = np.stack([P for _ in range(e)], axis=0)
9
10    H = np.zeros((e, f))
11    for n in np.arange(w, f):
12        m = n - w + 1
13        Cn = X[:, :, m:(n+1)]
14        M = 1.0 / (np.mean(Cn, axis=2) + eps)
15        M = np.expand_dims(M, axis=2)
16        Cn = np.multiply(Cn, M)
17
18        S = np.dot(Q, Cn)
19        S = S[0, :, :, :]
20        S = np.swapaxes(S, 0, 1)
21
22        S1 = S[:, 0, :]

```

```

23     S2 = S[:, 1, :]
24     alpha = np.std(S1, axis=1) / (eps + np.std(S2, axis=1))
25     alpha = np.expand_dims(alpha, axis=1)
26     Hn = np.add(S1, alpha * S2)
27     Hnm = Hn - np.expand_dims(np.mean(Hn, axis=1), axis=1)
28
29     H[:, m:(n + 1)] = np.add(H[:, m:(n + 1)], Hnm)
30
31     return H

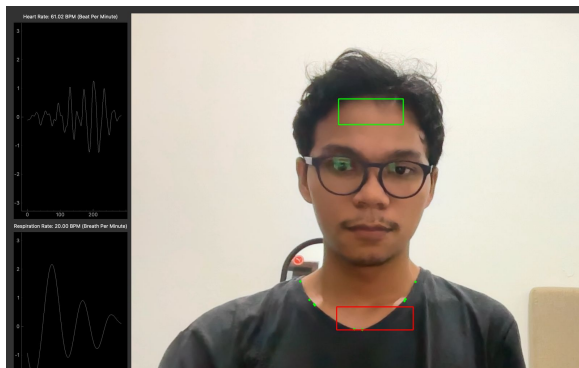
```

Kode 22: Fungsi cpu_POS

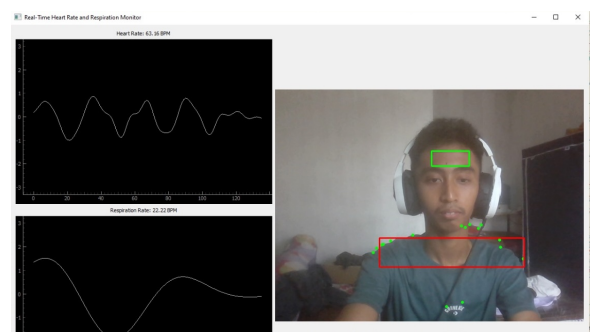
Fungsi di atas ini menggunakan algoritma *Plane Orthogonal-to-Skin* (POS) untuk menghitung sinyal rPPG atau jumlah detak jantung pengguna.

4 Hasil

Berdasarkan hasil implementasi, program berhasil mendeteksi wajah dan bahu dengan menggunakan MediaPipe.



(a) Hasil Deteksi Program 1



(b) Hasil Deteksi Program 2

Gambar 1: Hasil Deteksi Program

Bounding box digunakan sebagai panduan bagi pengguna untuk menyesuaikan posisi agar dapat terdeteksi oleh program dengan baik. Fitur ini memberikan umpan balik visual yang membantu meningkatkan akurasi deteksi sekaligus mempermudah penggunaan.

Penggunaan optical flow pada kode respirasi memberikan efisiensi yang signifikan. Dengan tidak perlu melakukan re-deteksi pada setiap frame, proses pengolahan video menjadi lebih optimal dan lancar.

Pemilihan ROI (Region of Interest) untuk masing-masing fungsi juga berkontribusi besar terhadap performa program. Untuk rPPG (Remote Photoplethysmography), ROI dipilih di area sekitar jidat karena efektif dalam menangkap sinyal detak jantung. Sementara itu, untuk sinyal pernapasan, ROI didefinisikan sebagai bounding box di sekitar titik tengah antara landmark bahu (Landmark 11 dan 12 pada MediaPipe). Penempatan ini sangat efektif karena gerakan dada, yang menjadi indikator pernapasan, dapat terdeteksi dengan baik.

Selain itu, integrasi antarmuka pengguna grafis (GUI) yang dibangun dengan PyQt5 semakin meningkatkan pengalaman pengguna. GUI ini menampilkan live feed kamera bersama dengan visualisasi sinyal yang dihasilkan dan estimasi BPM (Breaths atau Beats per Minute) secara real-time. Informasi ini disajikan dengan cara yang intuitif dan mudah dipahami, sehingga program ini menjadi lebih informatif dan ramah bagi pengguna.

5 Kesimpulan

Program yang dikembangkan berhasil mendeteksi detak jantung (HR) dan sinyal pernapasan secara akurat dengan memanfaatkan teknik rPPG (Remote Photoplethysmography) dan MediaPipe. Penggunaan MediaPipe memungkinkan deteksi yang andal pada area wajah dan bahu sebagai titik referensi. ROI yang dipilih, yaitu di sekitar jidat untuk HR dan area di antara landmark bahu untuk sinyal pernapasan, memberikan hasil yang optimal.

Penggunaan optical flow dalam proses analisis sinyal respirasi memastikan efisiensi tinggi dengan mengurangi kebutuhan re-deteksi setiap frame, sehingga meningkatkan kinerja program secara keseluruhan. Selain itu, integrasi GUI berbasis PyQt5 memberikan kemudahan bagi pengguna dengan menampilkan live feed kamera, visualisasi sinyal, dan estimasi BPM (Breaths dan Beats per Minute) secara real-time dalam format yang informatif dan mudah dipahami.

Dengan memadukan teknik deteksi visual berbasis landmark dan pengolahan sinyal, program ini memberikan solusi yang efisien dan user-friendly untuk pengukuran HR dan respirasi berbasis gerakan tubuh.

References

- [1] W. Wang, A. C. Den Brinker, S. Stuijk, and G. De Haan, “Algorithmic principles of remote ppg,” *IEEE Transactions on Biomedical Engineering*, vol. 64, no. 7, pp. 1479–1491, 2016.
- [2] C. Massaroni, D. Lo Presti, D. Formica, S. Silvestri, and E. Schena, “Non-contact monitoring of breathing pattern and respiratory rate via rgb signal measurement,” *Sensors*, vol. 19, no. 12, p. 2758, 2019.